Experiment No. 8

Title

*A Search Algorithm Using Artificial Intelligence**

Aim

To develop a Python program that implements the A* Search Algorithm to find the shortest path between the source and destination.

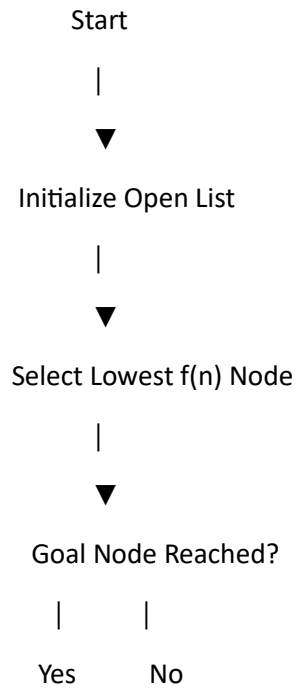
Objective

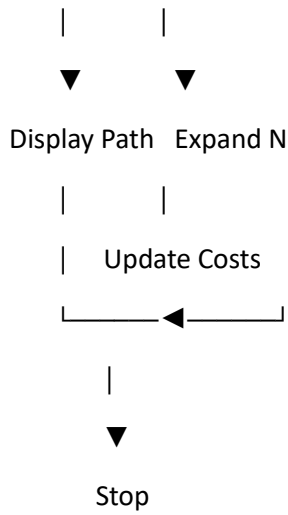
The objective of this experiment is to understand the A* Search Algorithm and implement it to find the optimal path using path cost and heuristic values.

Algorithm

Step 1: Define the graph and heuristic values.
Step 2: Initialize the Open List with the start node.
Step 3: Select the node with the lowest **f(n)** value.
Step 4: Expand its neighboring nodes.
Step 5: Update the path cost and heuristic values.
Step 6: Repeat until the goal node is reached.
Step 7: Display the shortest path and total cost.

Flowchart





Python Program

```
import heapq


graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 6)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}


heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 2,
    'G': 0
```

```
}

def astar(start, goal):
    pq = [(heuristic[start], 0, start, [start])]
    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)

        if node == goal:
            return path, g

        if node in visited:
            continue

        visited.add(node)

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                heapq.heappush(
                    pq,
                    (g + cost + heuristic[neighbor],
                     g + cost,
                     neighbor,
                     path + [neighbor])
                )

path, cost = astar('A', 'G')

print("Shortest Path:", path)
print("Total Cost:", cost)
```

Sample Output

Shortest Path: ['A', 'B', 'D', 'G']

Total Cost: 6

Result

The shortest path between the source and destination was successfully found using the A* Search Algorithm.

Conclusion

Thus, the A* Search Algorithm successfully determined the optimal path by combining the actual path cost and heuristic estimate, making it efficient for pathfinding problems.